Machine Learning Day 3

▼ Collecting Data

This is usually the next stage after formulating a problem. Before we talk about collecting data, let's even understand the meaning of data. According to wikipedia, "data are a set of values of qualitative or quantitative variables about one or more persons or objects."

There are 2 main types of data that are:

- Structured data that are organized in tabular or spreadsheet format. Example of tabular data includes customer records, car sales, etc...
- Unstructure data such as images, texts, sounds, and videos. Unstructured data are not organized as the former.

Nowdays, there are lots of open-source datasets on platforms like Kaggle, Google datasets, UCI, and government websites. So, if you are solving a problem that someone solved before, it's very likely that you will find it somewhere in those platforms, or other public sources. Your job as machine learning engineer is to find the relevant data that you can use to solve the presented problem.

That said, there are times that you will have to collect your own data, especially if you are solving a problem that no one solved before. In this case, consider the time that you will have to spend collecting data and the cost. You also do not need to wait until you have your desired data points before you can start. Embrace ML development early on so that you can learn if you (really) need more data. This idea is inspired by Andrew Ng.

Also, when collecting the data, quality is better than quantity. There are times where small data but good data can outwork big poor data. The amount of data you need is going to depend on the problem you're solving and its scope, but whatever the problem is, aiming to collect good data is the way to go.

▼ Establishing a Baseline

Without a benchmark, you won't know how to evaluate your results properly. A baseline is the simplest model that can solve your problem with minimal

requirements. It does not have to be a model. It can be an existing open source application, a statistical analysis or intuitions you get from data at a quick glance.

The single most purpose of a baseline is to act as a reference point when comparing the actual model with the baseline. The ultimate goal is to beat a baseline, and sometime, if you can't beat it, it might mean the project is not worth pursuing, or the baseline can be all you need.

▼ **Exploratory data analysis(EDA)**

Before manipulating the data, it is quite important to learn about the dataset. This can be overlooked, but doing it well will help you know the effective strategies to be applied while cleaning the data.

Go through some values, plot some features, and try to understand the correlation between them. If the work is vision-related, visualize some images, spot what’s missing in the images. Are they diverse enough? What types of image scenarios can you add? Or what are images that can mislead the model?

Here are other things that you would want to inspect in the data:

- Duplicate values and samples
- Corrupted or data with unsupported formats (ex: having an image of .txt and with 0 Kilobytes)
- Class imbalances
- Biases

Before performing EDA, split the data into the training set, validation and test sets to avoid data leakage. The training set is used for training the model, validation set is used for evaluating the model performance during training to suggest improvements, and test set is for evaluating the final and best model performance. Validation and test set should have the same statistical distribution.

Also, to avoid data leakage, do not touch the test set in EDA or anywhere before the model training.

▼ **Data Preprocessing**

Data processing is perhaps the biggest part of any machine learning project. There is a notion that Data Scientists and Machine Learning Engineers spend more than 80% of their time preparing the data and this makes sense because the real world data are messy.

In this step, it is where you convert the raw data to go in a format that can be accepted by the machine learning algorithms.

Data preprocessing is hard because there are different types of data and the way you process one is different to the other. For example, in structured data, the way you process numerical features is going to be different to categorical features. Also in unstructured data, the way you manipulate images is going to be different to how you manipulate texts or sounds.

As the next parts will cover the practical implementations of typical data preprocessing steps, let's be general about things you're likely going to deal with while manipulating the features:

- **Imputing missing values:** Missing values can either be filled, removed or left as they are. There are various strategies for missing values such as mean, median, frequent imputations, backward and forward fill, and iterative imputations. The right imputation technique depends on the problem and the dataset. With the exception of tree based models, most machine learning models do not accept missing values.
- **Encoding categorical features:** Categorical features are all types of features that have categorical values. For example, A gender feature having the values male and female is a categorical feature. You will want to encode such types of features. The techniques for encoding them are label encoding where you can assign 0 to Male and 0 to Female, or one hot encoding where you can get the binary representations (0s and 1s) in one hot matrix. You will see this later in practice.
- **Scaling the numeric features:** Most ML models work well when the input values are scaled to small values because they can train and converge faster than they would otherwise. There are two main scaling techniques that are normalization and standardization. Normalization rescales the feautures to the values between 0 and 1 whereas standardization rescales the features to have mean of 0 and a unit standard deviation. If you are

aware that your data has normal or gausian distribution, normalization can be a good choice. Otherwise, standardization will work well in many cases.

In many textbooks and courses, data preprocessing is also referred to data cleaning or data preparation. Feature engineering is also a part of data preprocessing. Feature engineering is a creative task and it require some extra knowledge about the data and the problem as it involves creating new features from existing features.

▼ Selecting and Training a Model

Selecting, creating, and training a machine learning model is the tiniest part in any typical machine learning workflow. There are different types of models but in broad, most of them falls into these categories: linear models such as linear and logistic regression, tree-based models such as decision trees, ensemble models such as random forests, and finally neural networks.

Depending on your problem, you can choose any relevant model from these categories or tries many of them. But, it is worth mentioning that getting a model that can ultimately solve a given problem is no *free lunch scenario*. You have to experiment with different models to get one that works for your problem and dataset.

To reduce your modelling curve, here are a few things that you can consider while choosing a machine learning model:

- **The scope of the problem:** The scope or type of your problem can give a strong signal on what learning algorithm to use. Take an example: If you are going to build an image classifier, neural networks (Convolutional Neural Networks specifically) might be your go to algorithm.
- **The size of the dataset:** Linear models tend to work well in small data problems, whereas ensemble models and neural network can work well when given huge amount of data.
- **The level of interpretability:** If you want the predictions of your model to be explainable, neural networks may not help. Tree based models such as decision trees can be explainable compared to other models.
- **Training time:** Complex models (neural networks and ensemble models) will take too long to train and thus draining the computation resources. On

the other hand, linear models can train faster.

As you can see, there is a trade-off during model selection. You want explainability, choose models that can provide that for you, most models don't. You have a small dataset or you care about the training time, same thing, a right model for you.

▼ **Performance Error Analysis**

Performing error analysis will guide you throughout the process of improving the results of the model. The improvement can either be from the data or the model.

One of the best way to do error analysis is to plot the learning curve and to try noticing where the model is failing and what might be the reason, and the right actions that you can take to reduce the errors.

To improve the model part, you can try different model configuration values or hyperparameters. You can also try different models to see one that works well. But also, good model comes from good data, so it's important to spend time examining the results of the model with respect to the input data.

Here are questions that you can yourself iteratively in the process error analysis:

- Is the model doing poorly on all classes or is it one specific class?
- Is it because there are not enough data points for that particular class compared to other classes?
- There are trade-offs and limits on how much you can do to reduce the errors. Is there a room for improvement?

Often, the improvements will not come from tuning the model, but spending time to increase the number of training samples and data quality.

When improving the data, you can create artificial data (a.k.a data augmentation). This will work well most of the time. The whole error analysis is an iterative process, keep doing it and always aim to improve the data than the model.